

1 操作系统主要的成就

- 进程 (Processes)
- 内存管理 (Memory Management)
- 信息保护和安全 (Information protection and security)
- 调度和资源管理 (Scheduling and resource management)
- 系统结构 (System structure)

2 现代操作系统的特征

微内核，多线程，对称多处理，分布式操作系统，面向对象设计

3 多道批处理系统

	批处理多道程序设计	分时
主要目标	最大化处理器使用率	最小化响应时间
操作系统指令源	作业控制语言 作业提供的命令	终端输入的命令

分时系统关注的是轮转时间，时间限制策略更有效是因为它给所有进程一个较短的处理时间。批处理系统关心的是吞吐量，更少的上下文转换和更多的进程处理时间。因此，最小的上下文转换最高效。

4 分时系统

- 使用多道程序设计来处理多个交互作业
- 多用户分享处理器时间
- 多用户通过终端同时访问系统
- 由于人的反应时间相对比较慢，因此分时系统的响应时间接近于专用计算机

5 进程

- 一个正在执行的程序
- 一个正在计算机中执行的程序实例
- 能分配给处理器并由处理器执行的实体
- 一个具有以下特征的活动单元：一组指令序列的执行、一个当前状态和相关的系统资源集合

6 进程基本元素

- 标识符 (Identifier)
- 状态 (State)
- 优先级 (Priority)
- 程序计数器 (Program counter)
- 内存指针 (Memory pointers)
- 上下文数据 (Context data)
- I/O状态信息 (I/O status information)
- 审计信息 (Accounting information)

7 进程状态模型

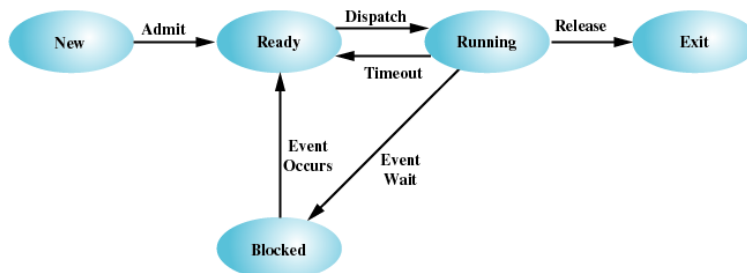
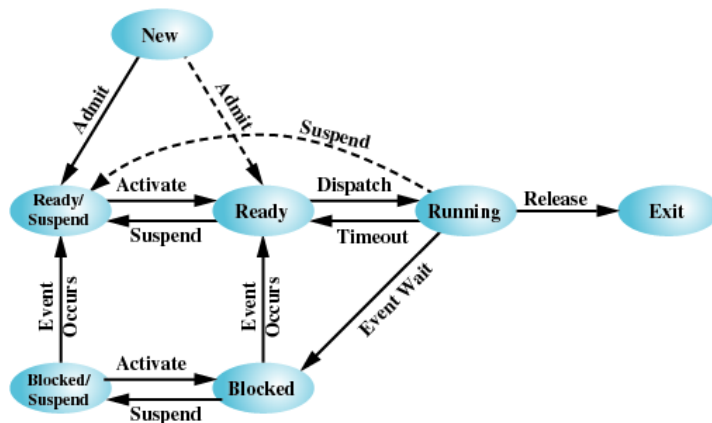


Figure 3.6 Five-State Process Model



(b) With Two Suspend States

Figure 3.9 Process State Transition Diagram with Suspend States

5 状态模型：运行态 (Running)，就绪态 (Ready)，阻塞态 (Blocked)，新建态 (New)，退出态 (Exit)

7 状态模型：运行态 (Running)，就绪态 (Ready)，阻塞态 (Blocked)，新建态 (New)，退出态 (Exit)

,就绪挂起态 (Ready Suspend)，阻塞挂起态 (Blocked Suspend)。

主要优势：1. 增加了就绪挂起和阻塞挂起可以有效进行操作系统的中层调度，控制多道程序设计的程度。 2.两个挂起态可释放内存空间缓解内存不足的问题。

8 进程切换

进程状态切换步骤

保存处理器上下文，包括程序计数器和其他寄存器
更新当前处于运行态的进程的进程控制块
将进程控制块移至相应的队列（就绪、阻塞、就绪/挂起）
选择另一个进程执行
更新所选进程的进程控制块
更新内存管理数据结构
恢复被选进程的上下文信息

9 操作系统与用户进程的关系

- 无进程的内核
 - 在所有进程之外执行操作系统内核
 - 操作系统代码作为一个单独实体运行在特权模式
- 在用户进程中执行
 - 在用户进程的上下文中执行操作系统软件
 - 当执行到操作系统代码时，进程进入特权模式执行
 - 无需进程切换，只需模式切换
- 基于进程的操作系统
 - 操作系统作为一组系统进程来实现
 - 模块化操作系统
 - 在多处理器和多机的环境中十分有用

10 线程

线程相关数据

- 执行状态
- 在未运行时保存线程上下文
- 执行栈
- 用于每个线程的局部变量和静态存储空间
- 对于所属进程的内存和资源的访问
 - 进程中的所有线程共享这些资源

11 对称多处理

- 内核可以在任何处理器上执行
- 每个处理器从可用的进程或线程池中进行各自的调度

12 微内核

- 小型的操作系统核心
- 仅仅包含基本的核心操作系统功能
- 许多传统上包含在操作系统中的服务现在作为外部子系统
 - 设备驱动程序
 - 文件系统
 - 虚拟内存管理
 - 窗口系统
 - 安全服务

微内核组织结构的优点

- 一致性接口
 - 不需要区分是内核级服务还是用户级服务
 - 所有服务都是通过消息传递的方式提供
- 可扩展性
 - 允许增加新服务
- 灵活性
 - 增加新功能
 - 删减现有功能

- 可移植性
 - 系统移植到新处理器上的改动仅仅需要改动微内核，而不需要改动其他的服务
- 可靠性
 - 模块化设计
 - 小的微内核可以被严格地测试

- 分布式系统支持
 - 消息的发送不需要知道目标机器是什么
- 面向对象操作系统支持
 - 组件是具有明确定义的接口对象，可以以搭积木的方式通过互连构造软件

13 线程与进程的关系

- 创建线程的时间比创建进程的时间少
- 终止线程的时间比终止进程的时间少
- 同一进程内的线程切换时间比进程切换时间少
- 由于同一进程内的线程共享内存和文件，因此同一进程内的线程间通讯不需要调用内核接口

进程状态改变对线程的影响

- 由于所有线程共享同一个地址空间，因此挂起一个进程将挂起该进程的所有线程
- 终止一个进程将终止该进程的所有线程

用户级线程状态和进程状态的关系

- 进程状态与线程状态两者之间不是一一对应关系
 - 线程阻塞不一定产生进程阻塞
- 线程是执行单元，进程是资源管理单元，因此某个线程的执行会使得进程状态变化

14 用户级线程和内核级线程

用户级线程的优点与缺点

- 优点
 - 线程切换不需要内核模式特权
 - 调用可以是应用程序专用的，使用定制的调度方式
 - 可以在任何操作系统中运行
- 缺点
 - 有可能引起阻塞，如：I/O调用
 - 纯粹的用户级线程策略不能利用多处理器技术

内核级线程的优点和缺点

- 优点
 - 内核可以同时把一个进程中的多个线程调度到多个处理器中
 - 线程阻塞不会引起进程阻塞
- 缺点
 - 线程切换需要内核模式特权

15 临界区

- 临界区
 - 是一段代码，在这段代码中进程将访问共享资源，当另外一个进程已经在这段代码中运行时，这个进程就不能在这段代码中执行。
 - 可以理解为一种代码的保护区，目的是为了使得两个进程不能同时访问共享资源

16 死锁

- 死锁
 - 两个或两个以上的进程因其中的每个进程都在等待其他进程做完某些事情而不能继续执行。
 - 互不相让

17 活锁

- 活锁
 - 两个或两个以上的进程为了响应其他进程中的变化而继续改变自己的状态但不做有用的工作。

18 互斥

- 互斥

- 当一个进程在临界区访问共享资源时，其他进程不能进入该临界区访问任何共享资源。

19 竞争条件

- 竞争条件

- 多个线程或者进程在读写一个共享数据时结果依赖于他们执行的相对时间。

20 饿死

- 饿死（饥饿）

- 一个可运行的进程尽管能继续执行，但被调度器无限期地忽视，而不能被调度执行的情况。

21 互斥的条件

互斥的条件

- 一次只允许一个进程进入临界区
- 一个在非临界区停止的进程必须不干涉其他进程
- 不会出现死锁或饿死
- 当没有进程在临界区时，任何需要进入临界区的进程必须能够立即进入
- 对相关进程的速度和处理器数据没有任何要求和限制
- 一个进程驻留在临界区中的时间必须是有限的

22 信号量

- 信号量是一种特殊的变量，用于发送信号
- 若一个进程正在等待一个信号，则该进程将被阻塞直到信号被发送

23 管程

- 管程是一个程序设计语言结构
- 管程的主要特征
 - 局部数据变量只能由管程访问
 - 进程通过调用管程中的一个过程进入管程
 - 每次只有一个进程在管程中执行

24 消息

- 作用

- 实施互斥
- 交换信息

- 消息原语

```
send (destination, message)  
receive (source, message)
```

25 生产者与消费者

- 有一个或多个生产者产生某种类型的数据并放置在缓冲区中
- 一个消费者从缓冲区中取数据，每次取一项
- 任何时候只有一个生产者或消费者能访问缓冲区

26 死锁原理

- 互斥
 - 每次只有一个进程使用一个资源
- 占有且等待
 - 当一个进程等待其他资源时，可以占有已分配的资源
- 非剥夺
 - 不可能从已经占有资源的进程手中强行剥夺资源

27 死锁预防

死锁预防：约束资源请求

- 互斥
 - 操作系统必须支持互斥
- 占有且等待
 - 要求进程一次性请求所有需要的资源
- 非剥夺
 - 进程在进一步申请其他资源被拒绝时，必须释放它最初的资源，并在合适的时候再次请求这些资源
 - 操作系统可以剥夺一个进程，要求它释放资源
- 循环等待
 - 定义资源类型的线性顺序，使得进程必须依照相同的顺序申请资源

19

28 死锁避免

死锁避免：预测死锁可能

- 死锁预防会导致低效的资源使用和进程执行
- 判断当一个资源分配决策被执行后，是否会导致死锁
- 需要关于将来的进程资源请求的知识

- 优点
 - 不需要剥夺和重新运行进程，比死锁预防限制少
- 限制
 - 必须事先声明每个进程请求的最大资源
 - 进程必须是无关的，没有任何同步要求限制
 - 分配的资源数目必须是固定的
 - 进程占有资源时不能退出

29 死锁检测

死锁检测

- 无需最大资源声明
- 使用类似死锁避免中的安全测试算法
 - 检测执行分配后是否安全
 - 已分配资源的进程优先
 - 当算法执行完成后存在未标记的进程，则产生死锁
- 死锁检测不能保证防止死锁，仅仅是检测当前是否存在死锁

30 哲学家就餐问题

31 内存分区

固定分区

- 等大小分区
 - 小于或等于分区大小的任何进程都可以装入到任何可用分区中
 - 若所有分区已满，并且没有进程处于就绪态或运行态，则操作系统可以换出一个进程的所有分区
 - 难点：
 - 程序太大而不能放到一个分区中。程序员必须使用覆盖技术设计程序。
 - 主存利用率低。小程序可以占用一整个分区，容易产生内部碎片

- 由于分区固定，因此限制系统中活动进程的数目
- 小作业不能有效地利用分区空间

放置算法

- 等大小分区
 - 因为所有分区大小相等，所以使用哪个可用分区都没有关系
- 非等大小分区
 - 每个进程指定到足够容纳它的最小分区
 - 每个分区维护一个调度队列
 - 使每个分区内部浪费的空间最少

动态分区

- 分区长度和数目可变
- 进程分配与所需空间相等的存储空间
- 内存中产生空洞，即外部碎片
- 必须使用压缩技术移动进程使其占有连续空间

固定分区和动态分区的缺陷

- 固定分区限制了活动进程数目，并且内存空间利用率低
- 动态分区的维护需要进行压缩，增加开销

32 分页

分页

- 内存被划分为较小的固定大小的块，同时进程也被分成同样大小的块
- 进程中的块被称为页，内存中的块被称为帧（页框）
- 操作系统为每个进程维护一张页表
 - 进程的每一页对应的帧位置
 - 每个逻辑地址包含一个页号和在该页中的偏移量

分页与固定分区的区别

- 分区小
- 一个程序可以占据多个分区
- 分区不需要连续

33 分段

分段

- 所有程序的所有段的长度不需要相同
- 有最大段长度限制
- 地址包含两部分：段号和偏移量
- 分段类似于动态分区

34 放置算法

放置算法

- 操作系统必须确定分配哪个空闲块
- 最佳适配算法
 - 选择与请求大小最接近的块
 - 性能最差
 - 因为每次查找满足要求的最小块，所以使得剩下的碎片更小
 - 内存压缩必须经常执行
- 首次适配算法
 - 从开始扫描内存，选择大小足够的第一个可用块
 - 通常是最快和最好的
 - 使得内存前端出现很多小的空闲分区，并且每次查找时都要经过这些分区

- 邻近适配算法

- 从上一次放置的位置开始扫描内存，选择下一个大小足够的可用块
- 通常比首次适配算法差
- 常常导致在内存的末尾分配空间
- 使得位于内存末尾的最大空闲块很快被分裂成小块
- 需要内存压缩在内存末尾获得较大的内存块

35 伙伴系统

伙伴系统

- 整个空间被看做为一个大小为 2^U 的块
- 若请求的大小 s 满足 $2^{U-1} < s \leq 2^U$, 则分配整个空间
- 否则，该块被分成两个大小相等的伙伴
- 该过程一直继续到产生大于或等于 s 的最小块，并分配给该请求

36 逻辑地址向物理地址转换方式

37 两级页表

分页

- 每个进程都有自己的页表
- 页表项包含与主存中的页相对应的帧号
- 每个页表项需要有一位来表示它所对应的页当前是否在主存中
- 每个页表项需要有一位来表示页面内容是否被修改，若页面没有被修改，则页面换出时不需要写入磁盘

页表

- 整个页表占据大量的主存空间
- 页表大小与虚拟地址空间大小成正比
- 页表采用虚存方案
- 当一个进程运行时，一部分页表被载入主存

38 反向页表

反向页表

- 应用于PowerPC, UltraSPARC, and IA-64 体系结构中
- 虚拟地址的页号部分使用散列函数映射到哈希表中
- 哈希值指向反向页表
- 无论有多少进程，页表只需要实存中的一个固定部分

反向页表内容

- 页号
- 进程标识符
- 控制位
- 链指针

39 分页与分段结合

页尺寸的设计因素

- 页越小，内部碎片的总量越少
 - 页越小，每个进程需要的页的数目越多，将产生更大的页表
 - 大页表意味着大部分的页表存放在虚存而不是主存中
 - 辅存希望页尺寸大，从而实现更有效的数据块传送
- 页尺寸小，主存中有更多的页
 - 执行一段时间后，内存中的页包含最近访问的部分，因此，页错误率低
 - 增加页尺寸，引起页面包含的位置与最近访问的位置远，页错误率升高

分段的优势

- 大小不等，动态
- 简化不断增长的数据结构的处理
- 允许程序独立地改变或重新编译
- 有助于进程间的共享
- 有助于保护

分段与分页的结合

- 分页对程序员透明，消除了外部碎片
- 分段对程序员可见，具有处理不断增长的数据结构的能力以及支持共享和保护的能力
- 每个段被划分为固定大小的页

40 读取策略

读取策略

- 确定一页何时读入内存
- 请求式分页：当访问到某页中的一个单元时才将该页取入内存
 - 进程第一次启动时出现大量的页错误
- 预约式分页：一次读取更多的页
 - 读取被连续存储在辅存中的页更为有效

41 放置策略

放置策略

- 决定一个进程块驻留在实存中的什么位置
- 在分段系统中比较重要
- 在分页或组合方式的系统中，由于地址转换硬件和主存访问硬件可以有效访问任何页帧，所以不需要放置策略

42 替换策略

替换策略

- 哪一页被替换？
- 被替换页必须是在最近不可能被访问的页
- 大多数的策略基于最近的访问历史来预测将来的访问行为

- 帧锁定

- 若帧被锁定，则不能被替换
- 在锁定帧中的数据
 - 操作系统内核
 - 控制结构
 - I/O 缓冲
- 锁定帧通过帧关联一个lock位实现

基本替换算法

- 最佳策略
 - 选择替换下次访问距当前时间最长的页
 - 不可能完美预知将来的事件
- 最近最少使用策略(LRU)
 - 替换主存中上次使用距当前最远的页
 - 依据局部性原理，该页是最近最不可能访问到的页
 - 每一页增加一个最后一次访问的时间标签，更新和访问开销较大。
- 先进先出策略(FIFO)
 - 把分配给进程的页帧看做是一个循环缓冲区
 - 按照循环方式移除页面
 - 实现起来最简单的替换策略
 - 替换驻留在内存中时间最长的页（尽管该页可能经常被使用）
 - 被替换的页可能很快被用到
- 时钟策略
 - 帧关联一个使用位
 - 当某一页首次装入内存时，使用位置1
 - 当页被访问时，使用位置1
 - 当需要替换一页时，第一个使用位为0的帧被替换
 - 在搜索替换页时，每个使用位为1的帧被置为0

基本替换算法

- 页缓冲
 - 被替换页没有被丢弃，而是分配到以下两个表之一
 - 空闲页表（若未被修改）
 - 修改页表
 - 替换页仍然在主存中，便于再次被访问
 - 成簇写回

43 驻留集管理

驻留集大小

- 固定分配策略
 - 分配固定数目的帧用于进程执行
 - 当产生页错误，其中一页必须被替换
- 可变分配策略
 - 允许分配给一个进程的页帧在该进程的生命周期中不断地发生变化

固定分配、局部替换

- 事先确定分配给进程的总帧数
- 若分配过小，则会产生很高的页错误率
- 若分配过大，则主存中仅有少量程序，不利于提高处理器利用率

可变分配、全局替换

- 最易于实现，被许多操作系统采用
- 操作系统维护一个空闲帧表
- 当发生一次页错误时，一个空闲帧被添加到进程的驻留集中
- 若没有空闲帧，选择当前主存中的一帧替换（可能是其他进程的帧）
- 无法保证驻留集减少的进程适合于被替换

- 当一个新进程被装入主存时，依据应用类型、程序请求和其他原则分配页帧
- 当页错误发生时，选择该进程驻留集中的一页替换
- 不时地重新评估进程的帧分配情况，并进行相应调整

44 清除策略

清除策略

- 确定何时将一个被修改页写回辅存
- 请求式清除
 - 当一页被选择替换时才写回辅存
- 预约式清除
 - 成批写回
- 通过页缓冲技术清除
 - 维护修改页和未修改页两个列表
 - 修改页列表周期性写回辅存

45 加载控制

加载控制

- 确定驻留在主存中的进程数，即多道程序设计级
- 过少进程情况下，所有进程处于阻塞态的概率较大，因而时间大部分花费在交换上
- 过多进程情况下，进程驻留集不足，因而经常发生页错误，导致抖动

46 调度类型

- 长程调度
 - 加入到待执行的进程池。进程创建过程
- 中程调度
 - 加入到部分或者全部在主存中的进程集合中。进程由磁盘到主存
- 短程调度
 - 哪一个可用进程将被处理器执行。就绪到执行
- I/O调度
 - 哪一个进程挂起的I/O请求将被可用的I/O设备处理

长程调度

- 决定哪一个程序可以进入到系统中被处理
- 控制多道程序设计的程度
- 两个决策
 - 何时接纳一个或多个进程？多道程序设计的程度
 - 接受哪个作业或哪些作业？多种原则，如：先来先服务、优先级、期待执行时间和I/O需求等。

中程调度

- 是交换功能的一部分
- 基于管理多道程序设计的程度需求

短程调度

- 常常被称为分派器
- 执行最频繁
- 当有事件发生时执行
 - 时钟中断
 - I/O 中断
 - 操作系统调用
 - 信号

短程调度准则

- 面向用户
 - 响应时间
 - 从提交一条请求到响应出现在输出中所经历的时间间隔
- 面向系统
 - 处理器的使用效果和效率

- 与性能相关
 - 定量
 - 直接相关
 - 响应时间和吞吐量
 - 定性
 - 与性能无关
 - 可预测性

- 面向用户，与性能无关
 - 周转时间
 - 进程从提交到完成之间的时间间隔
 - 响应时间
 - 提交一个请求到开始接收响应之间的时间间隔
 - 最后期限
 - 指定进程完成的最后期限

- 面向用户，其他
 - 可预测性
 - 一个给定的工作运行的总时间量和总代价相同，即系统稳定，不出现大的波动

- 面向系统，与性能相关

- 吞吐量
 - 每个时间单位完成的进程数
- 处理器使用率
 - 处理器忙的百分比

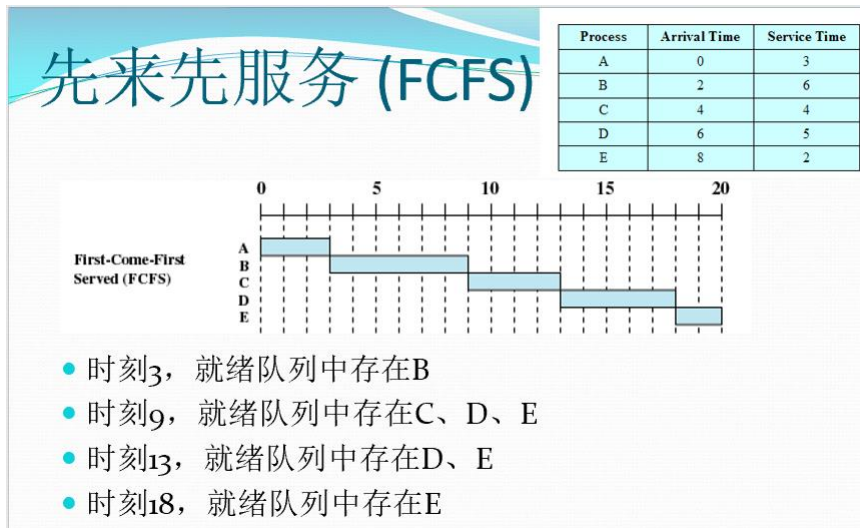
- 面向系统，其他

- 公平
 - 进程平等对待，不会饿死
- 强制优先级
 - 优先选择高优先级
- 平衡资源
 - 保持系统中所有资源忙

47 调度算法

先来先服务 (FCFS)

- 每个进程就绪后加入就绪队列
- 每次选择在就绪队列中存在时间最长的进程运行



先来先服务(FCFS)

- 执行长进程比执行短进程更好
- 短进程在执行前可能需要等待比较长的时间
- 偏向于受处理器限制的进程
 - 受I/O限制的进程必须等待受处理器限制的进程执行完成

进程	到达时间	服务时间 (T_s)	开始时间	结束时间	周转时间 (T_t)	T_t/T_s
W	0	1	0	1	1	1
X	1	100	1	101	100	1
Y	2	1	101	102	100	100
Z	3	100	102	202	199	1.99
平均值					100	26

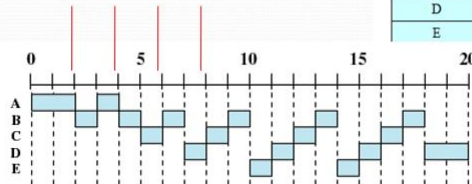
轮转(RR)

- 基于时钟的抢占策略
- 每个进程在被抢占前都给定一片时间执行

轮转(RR)

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round-Robin
(RR), $q = 1$



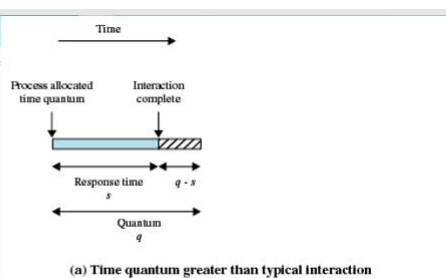
- 时刻2，就绪队列中存在B
- 时刻4，就绪队列中存在B、C
- 时刻6，就绪队列中存在B、D
- 时刻8，就绪队列中存在C、B、E

轮转(RR)

- 以一个周期性间隔产生时钟中断
- 当时钟中断发生时，当前正在运行的进程被置于就绪队列中
 - 选择下一个就绪作业
- 也被称为时间片

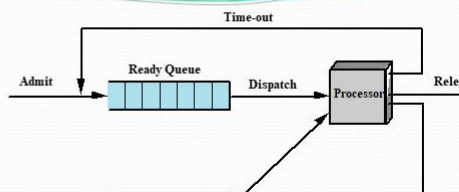
轮转(RR)

时间片长度问题
 过短：切换开销大
 过长：退化为FCFS
 略大于一次典型的交互操作所需时间



轮转(RR)

虚轮转法
 辅助队列优先于就绪队列
 直接转入运行态，解决受
 I/O限制的进程性能低

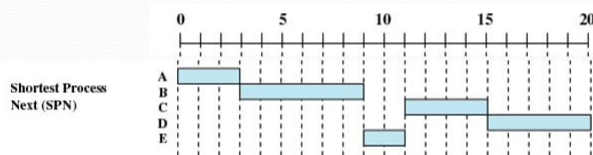


最短进程优先(SPN)

- 非抢占策略
- 选择所需处理时间最短的进程
- 短进程将会超过长作业，跳到队列头

最短进程优先(SPN)

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



- 时刻3，就绪队列中存在B[6]
- 时刻9，就绪队列中存在E[2]、C[4]、D[5]
- 时刻11，就绪队列中存在C[4]、D[5]
- 时刻15，就绪队列中存在D[5]

最短进程优先(SPN)

- 需要知道或估计每个进程所需的处理时间
- 若估计时间低于实际运行时间，系统可能终止该作业
- 对长进程可能饿死

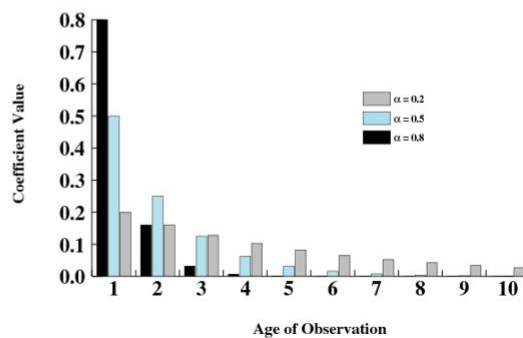
最短进程优先(SPN)

处理时间估算：指数平均法

$$S_{n+1} = \alpha T_n + (1-\alpha)\alpha T_{n-1} + \dots + (1-\alpha)^i \alpha T_{n-i} + \dots + (1-\alpha)^n S_1$$

S为估计值
T为进程执行时间

α 值越大，给较近观测值的权值越大

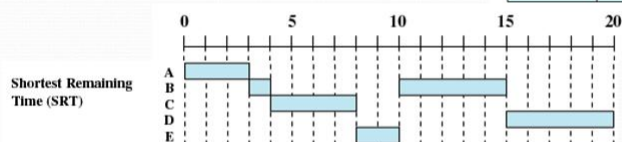


最短剩余时间(SRT)

- 针对SPN增加了抢占策略
- 必须估计处理时间

最短剩余时间(SRT)

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



- 时刻3, 就绪队列中存在B[6]
- 时刻4, 就绪队列中存在C[4]、B[6-1]
- 时刻8, 就绪队列中存在E[2]、B[6-1]、D[5]

最短剩余时间(SRT)

- 对长进程不利
- 可能产生饿死

最高响应比优先(HRRN)

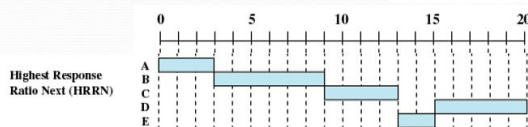
- 选择周转时间和实际服务时间比率最高的进程

$$R = \frac{w + s}{s}$$

- 当偏向短进程时, 长进程由于等待时间的增加, 提高了比值

最高响应比优先(HRRN)

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



- 时刻3时, 就绪队列中存在B[(1+6)/6]
- 时刻9时, 就绪队列中存在C[(5+4)/4]、E[(1+2)/2]、D[(3+5)/5]
- 时刻13时, 就绪队列中存在E[(5+2)/2]、D[(7+5)/5]
- 时刻15时, 就绪队列中存在D[(9+5)/5]

最高响应比优先(HRRN)

- 比较好地平衡了短进程和长进程

48 公平共享调度

公平共享调度

- 用户程序以一组进程或线程的方式运行
- 用户关心构成应用程序的一组进程的执行性能
- 需要基于进程集合做调度决策

49 多处理器调度

多处理器调度相关问题

- 将进程分配到处理器中
 - 分配问题
- 在单处理器上使用多道程序设计
 - 注重多处理器的分配，而不是单处理器忙
- 一个进程的实际分派
 - 进程调度算法

处理器分配方式

- 将处理器作为资源池并按要求把进程分配到相应的处理器中
- 静态分配：进程一直分配同一个处理器
 - 称为组调度策略
 - 为每个处理器维护一个短程队列
 - 优点：开销小
 - 缺点：一个处理器空闲而另一个处理器忙
- 动态分配：全局队列
 - 进程可以调度到任何一个可用处理器中

处理器分配体系结构

- 主从式
 - 核心功能总是运行在一个特定的处理器中
 - 主处理器负责调度
 - 从处理器发送请求给主处理器
 - 缺点
 - 主处理器故障将导致系统失败
 - 主处理器可能成为性能瓶颈
- 对等式
 - 操作系统可以运行在任何处理器中
 - 每个处理器进行自调度
 - 增加了操作系统的复杂性

50 多处理器调度与单处理器调度的区别

51 实时调度

实时调度

- 静态表驱动法
 - 确定在运行时一个任务何时必须开始执行
- 静态优先级驱动抢占法
 - 使用传统的优先级驱动的抢占式调度器
- 基于动态规划调度法
 - 运行时动态地确定可行性
- 动态尽力调度法
 - 不执行可行性分析

52 实时系统基本要求

实时操作系统的要求

- 可确定性
- 可响应性
- 用户控制
- 可靠性
- 故障弱化操作

53 时限调度

时限调度

- 实时应用程序不关注运行速度，关注最有价值的时间完成任务
- 常用信息
 - 就绪时间
 - 启动最后期限
 - 完成最后期限
 - 处理时间
 - 资源需求
 - 优先级
 - 子任务结构

时限调度

- 周期性任务
 - 任务描述为“每隔周期T一次”
- 非周期性任务
 - 有一个必须结束或开始的最后期限

54 优先级逆转

优先级逆转

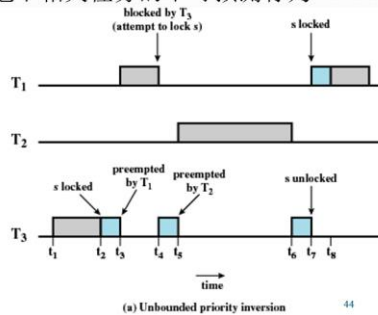
- 可能发生在任何基于优先级的可抢占调度方案中
- 当系统环境迫使一个较高优先级的任务去等待一个较低优先级任务时发生

无界限优先级逆转

- 优先级逆转的持续时间不仅依赖于处理共享资源的时间，还依赖于其他不相关任务的不可预测行为

优先级 $T_1 > T_2 > T_3$
 T_1 和 T_3 共享同一个资源

在一般的优先级逆转中
 T_1 的时间依赖于 T_3 ，但
 在这个例子中还依赖于
 T_2



55 磁盘调度算法

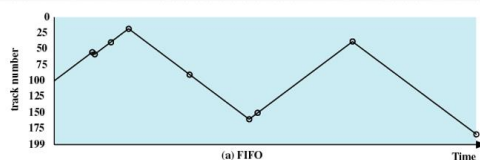
磁盘调度策略

- 寻道时间是产生性能差异的原因
 - 例子：书中P319
 - 紧凑存储，读取文件的时间0.064s
 - 随机存储，读取文件的时间20.04s
- 对于一个磁盘，存在大量的I/O请求
- 若请求式随机选择磁道，则磁盘I/O的性能非常低

磁盘调度策略

- 先进先出(FIFO)
 - 按顺序处理队列中的项目
 - 对所有进程公平
 - 若有大量进程竞争，则性能上接近于随机调度

请求序列：55，58，39，18，90，160，150，38，184



磁盘调度策略

- 优先级
 - 目标并不是优化磁盘使用率
 - 短的批作业和交互作业有较高的优先级
 - 提供比较好的交互响应时间

后进先出

- 对于事务处理系统有效
 - 设备资源提供给最近的用户，导致磁头移动少（局部性原理）
- 可能会出现饿死，由于无法获得队列头位置

- 最短服务时间优先（**Shortest Service Time First**）
 - 选择使磁头从当前位置开始移动最少的磁盘I/O请求
 - 总是选择最小寻道时间（不保证平均寻道时间最小）

请求序列：55， 58， 39， 18， 90， 160， 150， 38， 184

- **SCAN**

- 磁头仅仅沿一个方向移动，并在途中满足所有未完成的请求，直到到达这个方向的最后一个磁道或这个方向没有请求
- LOOK策略，反转服务方向
- 偏爱最靠里或最靠外的磁道作业

请求序列：55， 58， 39， 18， 90， 160， 150， 38， 184

- **C-SCAN**

- 扫描限定在一个方向
- 当访问到沿某个方向的最后一个磁道时，磁头返回到磁盘相反方向磁道的末端，并再次开始扫描

请求序列：55， 58， 39， 18， 90， 160， 150， 38， 184

- **N-step-SCAN**

- 磁盘请求队列分成长度为N的子队列
- 每次使用SCAN处理一个子队列
- 新请求加入到其他队列（非正在处理的队列）

- **FSCAN**

- 两个队列
- 一个队列用于新请求

56 RAID 各个级别

RAID

- 独立磁盘冗余阵列（**Redundant Array of Independent Disks**）
- 操作系统把一组物理磁盘驱动器看做一个单独的逻辑驱动器
- 数据分布在物理驱动器阵列中
- 冗余的磁盘容量保存奇偶校验信息，从而保证当一个磁盘失败时，数据具有可恢复性

RAID 0 (无冗余)

- 降低成本
- 具有一定的并行，读写速度高

RAID 1 (镜像)

- 复制数据实现冗余
- 成本高
- 具有一定的并行，读取接近单个磁盘的两倍，写请求与单个磁盘相同

RAID 2 (通过Hamming码冗余)

- 可以纠正一位错误并检测双位错误
- 成本高
- 具有一定的并行，读写接近单个磁盘的两倍

RAID 3 (交错位奇偶校验)

- 通过异或重构数据
- 具有一定的并行，读写接近单个磁盘的两倍

RAID 4 (块奇偶校验)

- 通过异或重构数据
- 具有一定的并行，读时与RAID0相近，写时慢于单个磁盘

RAID 5 (块分布奇偶校验)

- 通过异或重构数据，奇偶校验条带分布在所有磁盘中
- 具有一定的并行，读时与RAID0相近，写时慢于单个磁盘

RAID 6 (双重冗余)

- 通过异或重构数据，两种数据校验算法
- 数据可用性高，三个磁盘失效时才丢失数据
- 具有一定的并行，读时与RAID0相近，写时慢于RAID5

57 文件系统软件结构

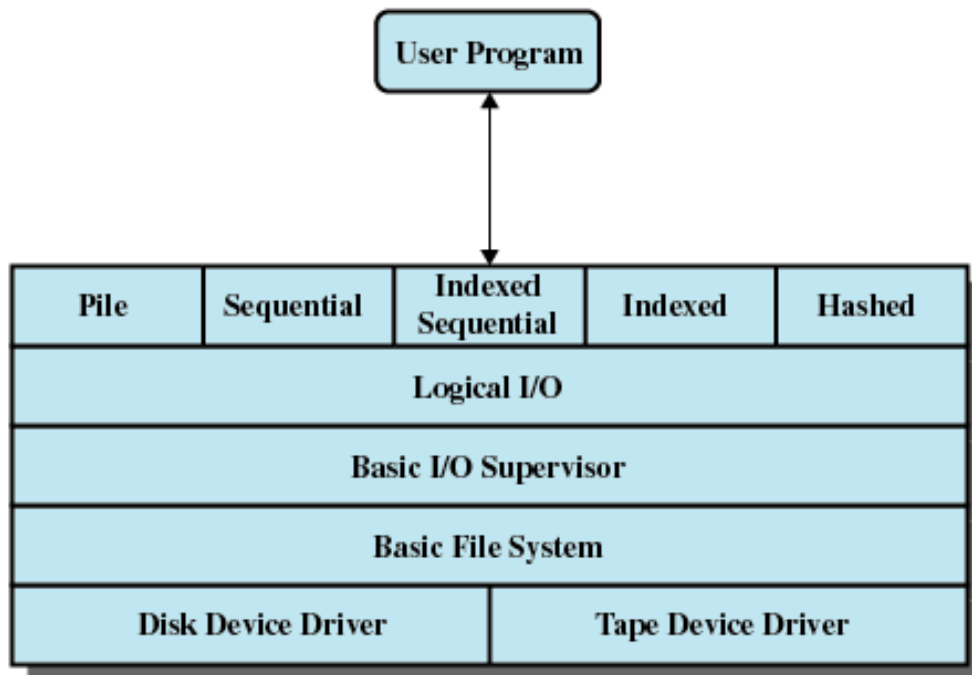


Figure 12.1 File System Software Architecture

58 文件组织类型

• 堆

- 数据按到达的顺序被采集
- 目的是积累大量数据并保存
- 记录可以有不同域
- 没有结构
- 记录访问通过穷举搜索方式

- 顺序文件

- 记录使用固定格式
- 记录长度相同
- 每个记录的对应域一样 (顺序和长度)
- 域名和长度是文件的属性
- 其中一个域是关键域
 - 唯一标识记录
 - 记录按照关键域的顺序存储
- 新记录放置在日志文件或事务文件中
- 成批更新用于将日志文件和主文件进行合并

- 索引顺序文件

- 索引提供快速接近目标记录的查找能力
 - 包含关键域和指向主文件的指针
 - 首先, 搜索索引, 查找关键域值等于目标关键域值或者位于目标关键域值之前且最大的索引
 - 其次, 搜索主文件, 在索引指针指向的位置开始搜索

- 顺序文件与索引顺序文件比较

- 例子: 一个文件包含100万条记录
- 在顺序文件中搜索一条记录的平均次数为 500,000
- 若一个索引包含1000 项, 索引中搜索平均需要 500 次, 主文件中搜索平均需要 500次。综合需要 1000 次

- 索引顺序文件

- 新记录被加入到溢出文件中
- 主文件中逻辑顺序位于新记录之前的记录修改其指针指向新记录
- 溢出文件在成批更新时与主文件合并
- 多级索引可以增加访问效率

- 索引文件

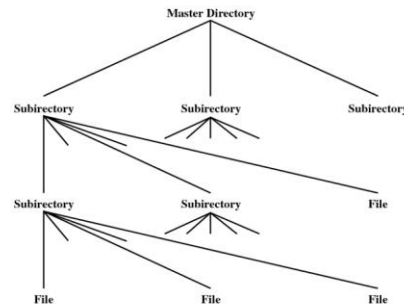
- 对不同的域使用多索引
- 完全索引: 包含主文件中每条记录的索引项
- 部分索引: 包含感兴趣域的记录的索引项

- 直接文件或散列文件

- 直接访问磁盘中任何一个地址已知的块
- 每条记录需要一个关键域

层次或树形目录结构

- 主目录下包含用户目录
- 每个用户目录可以具有子目录项和文件项



层次或树形目录结构

- 文件可以按照从根目录或主目录向下到各个分支，最终直到该文件的路径来定位
- 多个文件可以具有相同的文件名，只要保证它们的路径名唯一
- 当前目录作为工作目录
- 文件按照相对于工作目录的方式进行访问

记录组块

- 设计问题
 - 块长度是固定还是可变？
 - 块的相对大小是多少？
- 组块方法
 - 固定组块：记录长度固定的顺序文件最常用的方式
 - 可变长度跨越式组块：存储效率高，文件大小无限制，难以实现
 - 可变长度非跨越式组块：空间浪费，限制记录的大小不能超过块大小

- 连续分配
 - 在创建文件时，给文件分配一组连续的块
 - 文件分配表中每个文件只需要一项
 - 起始块和文件长度
- 会出现外部碎片
 - 需要执行压缩算法

- 链式分配
 - 分配基于单个块
 - 每个块包含一个指向下一块的指针
 - 文件分配表中只需要一个表项
 - 起始块和文件长度
 - 没有外部碎片
 - 适合于顺序文件
 - 局部性原理不再适用
-
- 索引分配
 - 每个文件在文件分配表中有一个一级索引
 - 分配给该文件的每个分区在索引中都有一个表项
 - 文件分配表包含索引块号

使用信号量互斥

```
/* program mutualexclusion */
const int n = /* number of processes */;
semaphore s = 1;
void P(int i)
{
    while (true)
    {
        semWait(s);
        /* critical section */;
        semSignal(s);
        /* remainder */;
    }
}
void main()
{
    parbegin (P(1), P(2), . . . , P(n));
}
```

Figure 5.6 Mutual Exclusion Using Semaphores

正确的实现

二元信号量方式

对于可能修改的变量，在互斥时往往增加一个变量保存修改前的值

思考：能不能将消费者中的semSignal(s)放在if(m==0)之后？

```
/* program producerconsumer */
int n;
binary_semaphore s = 1;
binary_semaphore delay = 0;
void producer()
{
    while (true)
    {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true)
    {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}
```

Figure 5.10 A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

正确的实现

一般信号量方式

信号量s用于保护对缓冲区的操作

信号量n用于统计可用资源数

```
/* program producerconsumer */
semaphore n = 0;
semaphore s = 1;
void producer()
{
    while (true)
    {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true)
    {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.11 A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores

正确的实现

一般信号量方式

信号量s用于保护对缓冲区的操作

信号量n用于统计可用资源数

信号量e记录空闲空间数

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1;
semaphore n = 0;
semaphore e = sizeofbuffer;
void producer()
{
    while (true)
    {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n)
    }
}
void consumer()
{
    while (true)
    {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.13 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores

使用管程解决生产者/消费者问题

```
/* program producerconsumer */
monitor boundedbuffer;
char buffer [N];
int nextin, nextout;
int count;
cond notfull, notempty;

/* space for N items */
/* buffer pointers */
/* number of items in buffer */
/* condition variables for synchronization */

void append (char x)
{
    if (count == N)
        cwait(notfull);
    buffer[nextin] = x;
    nextin = (nextin + 1) % N;
    count++;
    /* one more item in buffer */
    csignal(notempty);
}

void take (char x)
{
    if (count == 0)
        cwait(notempty);
    x = buffer[nextout];
    nextout = (nextout + 1) % N;
    count--;
    csignal(notfull);
}

{
    nextin = 0; nextout = 0; count = 0;
}
```

不需要对资源进行额外的保护
使用不灵活

使用管程解决生产者/消费者问题

```
void producer()
char x;
{
    while (true)
    {
        produce(x);
        append(x);
    }
}
void consumer()
{
    char x;
    while (true)
    {
        take(x);
        consume(x);
    }
}
void main()
{
    parbegin (producer, consumer);
}
```

Figure 5.16 A Solution to the Bounded-Buffer Producer/Consumer Problem Using a Monitor

52

使用消息进行互斥

```
/* program mutualexclusion */
const int n = /* number of processes */;
void P(int i)
{
    message msg;
    while (true)
    {
        receive (mutex, msg);
        /* critical section */;
        send (mutex, msg);
        /* remainder */;
    }
}
void main()
{
    create_mailbox (mutex);
    send (mutex, null);
    parbegin (P(1), P(2), . . . , P(n));
}
```

Figure 5.20 Mutual Exclusion Using Messages

使用消息解决生产者/消费者问题

消息即可以作为同步信号，同时又可以作为数据载体。

Mayproduce中的消息为同步信号作用
Mayconsume中的消息即作为同步信号，又作为数据载体

```
const int
capacity = /* buffering capacity */ ;
null = /* empty message */ ;
int i;
void producer()
{
    message pmsg;
    while (true)
    {
        receive (mayproduce, pmsg);
        pmsg = produce();
        send (mayconsume, pmsg);
    }
}
void consumer()
{
    message cmsg;
    while (true)
    {
        receive (mayconsume, cmsg);
        consume (cmsg);
        send (mayproduce, null);
    }
}

void main()
{
    create_mailbox (mayproduce);
    create_mailbox (mayconsume);
    for (int i = 1; i <= capacity; i++)
        send (mayproduce, null);
    parbegin (producer, consumer);
}
```

Figure 5.21 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Messages

方法1

读优先
一旦读进程获得信号量，则在所有读进程执行完毕前，写进程只能等待。

信号量wsem实施互斥
信号量x保护readcount

```
/* program readersandwriters */
int readcount;
semaphore x = 1, wsem = 1;
void reader()
{
    while (true)
    {
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer()
{
    while (true)
    {
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
    }
}

void main()
{
    readcount = 0;
    parbegin (reader, writer);
}
```

第一个读进程等待

所有读进程结束才释放信号

Figure 5.22 A Solution to the Readers/Writers Problem Using Semaphores: Readers Have Priority

方法2

写优先

当一个写进程需要写时，读进程将在信号量z上等待，使得写进程在其之前的读进程执行完毕后立刻获得执行。同时一旦写进程获得执行，在所有写进程执行完毕前，读进程无法执行

信号量wsem,rsem实施互斥

信号量x保护readcount

信号量y保护writecount

信号量z用于对读进程排队

```
/*program readersandwriters*/
int readcount, writecount;
semaphore x = 1, y = 1, z = 1, wsem = 1, rsem = 1;
void reader()
{
    while (true)
    {
        semWait (z);
        semWait (rsem);
        semWait (x);
        readcount++;
        if (readcount == 1)
            semWait (wsem);
        semSignal (x);
        semSignal (rsem);
        semSignal (z);
        READUNIT();
        semWait (x);
        readcount--;
        if (readcount == 0)
            semSignal (wsem);
        semSignal (x);
    }
}
void writer ()
{
    while (true)
    {
        semWait (y);
        writecount++;
        if (writecount == 1)
            semWait (rsem);
        semSignal (y);
        semWait (wsem);
        WRITEUNIT();
        semSignal (wsem);
        semWait (y);
        writecount--;
        if (writecount == 0)
            semSignal (rsem);
        semSignal (y);
    }
}
void main()
{
    readcount = writecount = 0;
    parbegin (reader, writer);
}
```

写进程进入读信号量排队，使得优先获得执行

所有写进程执行完毕后才释放